

Khwopa College of Engineering
OOP Lab Sheet
I Year/ I Part
Faculty: Computer and Electronics
Labsheet #5

Objectives:

1. Binary Operator Overloading and Unary Operator Overloading
2. Data Conversion: Basic to User-Defined Type, User-Defined Type to Basic Type, User-Defined to User-Defined Type

Tasks:

1. WAP to concatenate two user given string using the concept of Operator Overloading.
2. WAP to overload - - (pre decrement and post decrement both) operator.
3. WAP to create class named LandMeasure that have Ropani, Ana , Paisa and Dam as Data Members. Write a function to pass two object of type LandMeasure and return their sum using the Concept of Operator Overloading.
4. WAP that will add objects of a Date class with member variables day ,month and year using Operator Overloading.
5. WAP to convert currency from ruppee to dollar using both concept i.e. using constructor for data conversion as well as casting operator for data conversion.
6. WAP that converts object of Celsius type to object of Fahrenheit type.
7. WAP that will convert object from a class Rectangle to object of a class Polar using Casting Operator.

Binary Operator Overloading:

Operator + (e.g. S1+S2):

OO with Member Function:

```
#include <iostream>
using namespace std;

class Sample{
    int x ,y;
public:
    Sample():x(0),y(0){}
    Sample(int a, int b):x(a),y(b){}
    Sample operator +(Sample s){
        Sample temp;
        temp .x =x +s.x;
        temp .y=y+ s.y;
        return temp;
    }
    void display(){
        cout<<"Value of x: "<<x<<" and y: "<<y<<endl;
    }
};
```

```

int main(){
    Sample s1(2,3) ,s2(4,5) ,s3;
    s3=s1+s2; //s3=s1.operator+(s2)-->both are equivalent.
    s3.display();
    return 0;
}

```

OO with Friend Function:

```

#include <iostream>
using namespace std;
class Sample{
    int x, y;
public:
    Sample():x(0),y(0) {}
    Sample(int a, int b):x(a),y(b){}

    void display(){
        cout<<"Value of x: "<<x<<" and y: "<<y<<endl;
    }
    friend Sample operator +(Sample, Sample);
};
Sample operator +(Sample s11,Sample s22){
    Sample temp;
    temp.x =s11.x+s22.x;
    temp.y =s11.y+s22.y;
    return temp;
}
int main(){
    Sample s1(2,3), s2(4,5), s3;
    s3=s1+s2;
    s3.display();
    return 0;
}

```

Unary Operator Overloading:

Operator Negation(-) (e.g. -S):

OO with Member Function:

```

#include <iostream>
using namespace std;
class Sample{
    int x, y;
public:
    Sample():x(0),y(0){}
    Sample(int a ,int b):x(a),y(b){}
    Sample operator -(){
        Sample temp;
        temp.x=-x;
        temp.y=-y;
        return temp;
    }
}

```

```

        void display(){
            cout<<"Value of x: "<<x<<" and y: "<<y<<endl;
        }

};

int main(){
    Sample s1(2,3),s2;
    s2=-s1;//s2=s1.operator -();--> both are equivalent..
    s2.display();
    return 0;
}

```

OO with Friend Function:

```

#include <iostream>
using namespace std;
class Sample{
    int x, y;
public:
    Sample():x(0),y(0){}
    Sample(int a ,int b):x(a),y(b){}

    void display(){
        cout<<"Value of x: "<<x<<" and y: "<<y<<endl;
    }
    friend Sample operator -(Sample);
};

Sample operator -(Sample s){
    Sample temp;
    temp.x=-s.x;
    temp.y=-s.y;
    return temp;
}

int main(){
    Sample s1(2,3),s2;
    s2=-s1;
    s2.display();
    return 0;
}

```

Pre-increment (e.g. ++S):

OO with Member Function:

```
#include <iostream>
using namespace std;
class Sample{
    int x, y;
public:
    Sample():x(0),y(0){}
    Sample(int a, int b):x(a),y(b){}
    Sample operator ++(){
        Sample temp;
        temp.x=++x;
        temp.y=++y;
        return temp;
    }
    void display(){
        cout<<"Value of x: "<<x<<" and y: "<<y<<endl;
    }
};
int main(){
    Sample s1(2,3),s2;
    s1.display();
    s2=++s1;
    s2.display();
    return 0;
}
```

OO with Friend Function:

```
#include <iostream>
using namespace std;
class Sample{
    int x, y;
public:
    Sample():x(0),y(0){}
    Sample(int a ,int b):x(a),y(b){}

    void display(){
        cout<<"Value of x: "<<x<<" and y: "<<y<<endl;
    }
    friend Sample operator ++(Sample);
};
Sample operator ++(Sample s){
    Sample temp;
    temp.x=++s.x;
    temp.y=++s.y;
    return temp;
}
```

```

int main(){
    Sample s1(2,3),s2;
    s1.display();
    s2=++s1;
    s2.display();
    return 0;
}

```

Post-increment (e.g. S++):

OO with Member Function:

```

#include <iostream>
using namespace std;
class Sample{
    int x,y;
public:
    Sample():x(0),y(0){}
    Sample(int a, int b):x(a),y(b){}
    Sample operator ++(int){//Here,int is just a dummy..
        Sample temp;
        temp.x=x++;
        temp.y=y++;
        return temp;
    }

    void display(){
        cout<<"Value of x: "<<x<<" and y: "<<y<<endl;
    }
};

int main(){
    Sample s1(2,3),s2;
    s1.display();
    s2=s1++;
    s2.display();
    s1.display();
    return 0;
}

```

OO with Friend Function:

```

#include <iostream>
using namespace std;
class Sample{
    int x, y;
public:
    Sample():x(0),y(0){}
    Sample(int a, int b):x(a),y(b){}
    void display(){
        cout<<"Value of x: "<<x<<" and y: "<<y<<endl;
    }
    friend Sample operator ++(Sample &,int);
};

```

```

Sample operator ++(Sample &s ,int){
    Sample temp;
    temp.x=s.x++;
    temp.y=s.y++;
    return temp;
}
int main() {
    Sample s1(2,3),s2;
    s1.display();
    s2=s1++;
    s2.display();
    s1.display();
    return 0;
}

```

Data Conversion:

1. Basic to User-Defined Type:

//1.Basic/Primitive/In-built Type to User-Defined/Class Type..
 //--> Primitive type to Class/User-Defined type can be implemented
 // through Constructor.

```

#include <iostream>
using namespace std;

class Sample{
    int x ,y;
public:
    Sample(){x=0;y=0;}      //Default Constructor

    Sample(int m){
        x=y=m;
    }
    void display(){
        cout<<"Value of x is: "<<x<<" and y is: "<<y<<endl;
    }
};

int main(){
    int a=10;
    Sample s1;
    s1=a;    //Here, a is primitive type and s1 is class type.
    s1.display();
    return 0;
}

```

2. User-Defined to Basic Type:

```
//2. Class /User-Defined Type to Basic/Primitive Type..
//--> Class/User-Defined Type to Basic/Primitive Type can be
//      implemented through casting operator.
#include <iostream>
using namespace std;
class Sample{
    int x ,y;
public:
    Sample(int a, int b):x(a),y(b){}
    void display(){
        cout<<"Value of x is: "<<x<<" and y is:
            "<<y<<endl;
    }
    operator int(){    //casting operator..
        return int(x);
    }
};
int main(){
    int a;
    Sample s1(1,2);
    a=s1;              // Here, s1 is class type and a is
        primitive type.
    cout<<"Value of a is: "<<a<<endl;
    return 0;
}
```

3. User-Defined to User-Defined Type:

```
//3. User-Defined Type to User-Defined Type..
// To implement this kind of conversion, we can use either Constructor
// or casting operator..
```

1. Using Constructor

```
#include <iostream>
using namespace std;
class Product{
    int x ,y;
public:
    void input(int a, int b){
        x=a;
        y=b;
    }
    int getx(){
        return x;
    }
    int gety(){
        return y;
    }
}
```

```

};
class Label{
    int a, b;
    public:
        Label(){}
        Label(Product p){
            a=p. getX();
            b=p. getY();
        }
        void display(){
            cout<<"Value of a is: "<<a<<" and b is:
                "<<b<<endl;
        }
};
int main(){
    Label l1;
    Product p1;
    p1.input(1,2);
    l1=p1;           //one user-defined type to another user-
                    defined type conversion.
    l1.display();
    return 0;
}

```

//2. Using Casting operator..

```

#include <iostream>
using namespace std;
class Label{
    int a ,b;
    public:
        Label(int x=0,int y=0):a(x),b(y){}
        void display(){
            cout<<"Value of a is: "<<a<<" and b is: "<<b<<endl;
        }
};
class Product{
    int x ,y;
    public:
        void input(int a ,int b){
            x=a;
            y=b;
        }
        operator Label(){           //casting operator
            return Label(x, y);
        }
};

int main(){
    Label l1;

```



```
Product p1;  
p1.input(1,2);  
l1=p1;    //one user-defined type to another user-defined type  
         conversion.  
l1.display();  
return 0;  
}
```